Ex. No. 6

Development of Python Code Compatible with Multiple AI Tools

Aim

To write and implement Python code that integrates with multiple AI tools to automate tasks such as interacting with APIs, comparing outputs, and generating actionable insights.

Explanation

In this experiment, Python code is developed to demonstrate the integration of multiple AI tools. The program first analyzes a simulated AI-generated review using the **NLTK sentiment analysis tool**. Then, the program uses the **OpenAI API** to generate a smartphone review automatically. The generated review is analyzed again using the **NLTK Sentiment Intensity Analyzer** to determine whether the sentiment is positive, neutral, or negative.

Based on the sentiment score, the program produces actionable insights that can be used for decision-making, such as marketing promotion.

This experiment helps students understand how Python can interact with multiple AI tools and APIs to generate outputs, analyze results, and produce useful insights.

Code 1: Sentiment Analysis of AI-Generated Text

```
from nltk.sentiment import SentimentIntensityAnalyzer
import nltk

# Download sentiment model
nltk.download('vader_lexicon')

# Simulated AI-generated text
generated_text = "This smartphone offers outstanding battery life and an
```

```
intelligent AI camera that captures stunning photos."

print("Generated Review:\n")
print(generated_text)

# Sentiment analysis
sia = SentimentIntensityAnalyzer()
sentiment = sia.polarity_scores(generated_text)

print("\nSentiment Analysis:")
print(sentiment)

# Insight generation
if sentiment['compound'] > 0:
    print("\nInsight: The review is positive and suitable for marketing
promotion.")
else:
    print("\nInsight: The review tone is neutral or negative.")
```

Code 2: Integrating OpenAI API with Sentiment Analysis

```
from openai import OpenAI
from nltk.sentiment import SentimentIntensityAnalyzer
import nltk

# Download sentiment model
```

```
nltk.download('vader_lexicon')

# Initialize OpenAI client
client = OpenAI(api_key="YOUR_API_KEY")

# Persona Pattern Prompt
prompt = """
Act as a professional technology reviewer.
Write a short review about a new smartphone with a powerful battery and AI
camera.
"""

# Step 1: Generate text using the AI tool
response = client.responses.create(
    model="gpt-4.1-mini",
    input=prompt
)

generated_text = response.output[0].content[0].text

print("Generated Review:\n")
print(generated_text)

# Step 2: Sentiment analysis using another AI tool
sia = SentimentIntensityAnalyzer()
sentiment = sia.polarity_scores(generated_text)

print("\nSentiment Analysis:")
print(sentiment)

# Step 3: Generate simple insights
if sentiment['compound'] > 0:
    print("\nInsight: The product review is positive and suitable for marketing
promotion.")
else:
    print("\nInsight: The review tone is neutral or negative and may require
improvement.")
```

Sample Output

Generated Review:

This smartphone delivers excellent performance with its powerful battery and AI-powered camera. It captures clear photos even in low light and offers long-lasting battery life for everyday use.

Sentiment Analysis:

{'neg': 0.0, 'neu': 0.35, 'pos': 0.65, 'compound': 0.88}

Insight:

The product review is positive and suitable for marketing promotion.

Result

The Python program successfully integrated multiple AI tools. The **OpenAI API** generated a smartphone review, and the **NLTK sentiment analyzer** evaluated the sentiment of the generated text. The sentiment score indicated a positive review, and the system generated actionable insights suggesting that the product review could be used for marketing purposes. This experiment demonstrates how Python can integrate multiple AI tools and APIs to automate content generation, analyze outputs, and produce meaningful insights.